PROGRAMAÇÃO DE
COMPUTADORES I

# CONTROLE DE FLUXO
# LAÇOS DE REPETIÇÃO

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

# A LIMITAÇÃO DA EXECUÇÃO LINEAR

- Até o momento, os **algoritmos** construídos operam como uma via de **mão única**.
- A execução **flui** da linha **1** até o **final**, realizando no **máximo desvios** pontuais (através do comando **if**).
- Se uma mesma **ação** precisa ocorrer **3 vezes**, a ação no código **precisa** ser **repetida** 3 vezes.

# A LIMITAÇÃO DA EXECUÇÃO LINEAR

**Imagine** a necessidade de **exibir na tela** uma **contagem** de **1** a **10**. Seria viável escrever 10 comandos de print.
**Contudo**, se o projeto exigir o **processamento** de **10.000** registros, a **duplicação** manual de código torna-se **impraticável** e suscetível a **erros**.

# A LIMITAÇÃO DA EXECUÇÃO LINEAR

```
# O gargalo da repetição manual
print("Processando registro 1")
print("Processando registro 2")
print("Processando registro 3")
# Inviável para milhares de registros
```

# MATEMÁTICA E A SIMULAÇÃO DA REALIDADE

**Considere** uma **fábrica** que inicia sua operação produzindo **100 peças** no **primeiro mês** e **aumenta** essa capacidade em **20** peças a **cada ciclo** seguinte (uma Progressão Aritmética).

# MATEMÁTICA E A SIMULAÇÃO DA REALIDADE

Para **calcular** a produção **total** ao fim de **50 meses**, existe a **fórmula** matemática da **Soma dos Termos** de uma **P.A.**, que entrega o **resultado** de forma **direta** e elegante.

# MATEMÁTICA E A SIMULAÇÃO DA REALIDADE

E se o profissional **não** se **recordar** da **fórmula**?
Ou pior: e se **a regra** mudar no **meio** do **caminho**, com o aumento dependendo de **taxas variáveis** e não mais de uma **constante**? A **fórmula fechada** deixa de funcionar.

# MATEMÁTICA E A SIMULAÇÃO DA REALIDADE

O **processo iterativo** (calcular mês a mês). Na **ciência** e no **mercado**, existem **problemas** que são **naturalmente iterativos** — situações onde uma **equação exata** simplesmente **não existe** ou é **complexa** demais, como **simulações de clima** (meteorologia), **previsões** do **mercado financeiro** e **cálculos** de **aerodinâmica**. O computador **brilha** justamente em fazer esse "**trabalho** braçal" **rápido**.

# A LÓGICA HUMANA POR TRÁS DAS REPETIÇÕES

O **raciocínio humano** já lida **naturalmente** com **processos iterativos** baseados em **condições**, sem precisar de **fórmulas exatas**.

# A LÓGICA HUMANA POR TRÁS DAS REPETIÇÕES

Imagine a tarefa de **pregar** um **prego** na madeira. Não se **planeja** previamente um **número** exato de **marteladas** (ex: "darei exatamente 7 batidas"). A **lógica real** é: **bate-**se com o martelo **enquanto** o prego não estiver **totalmente fixado**.

# A LÓGICA HUMANA POR TRÁS DAS REPETIÇÕES

A **ação se repete**. Após cada **execução**, **avalia-se** a situação. Quando o **objetivo** é **atingido**, o **processo** é **interrompido**. O **código** de programação **precisa** desse **mecanismo** para resolver **problemas dinâmicos**.

# O CICLO DE VIDA DA REPETIÇÃO (INÍCIO, MEIO E FIM)

**Para** que um computador **repita** uma **tarefa** sem **travar** o sistema, devemos respeitar **três etapas lógicas** indispensáveis:

- **Início** (Ponto de Partida): A configuração do **estado inicial** antes da repetição **começar**. (Ex: O contador começa no Mês 1).

# O CICLO DE VIDA DA REPETIÇÃO (INÍCIO, MEIO E FIM)

**Para** que um computador **repita** uma **tarefa** sem **travar** o sistema, devemos respeitar **três etapas lógicas** indispensáveis:

- **Meio** (A Condição e a Ação): O **teste lógico** que **mantém** o ciclo **repetindo** ("Enquanto mês for <= 50") seguido do **trabalho** que **precisa** ser **feito**.

# O CICLO DE VIDA DA REPETIÇÃO (INÍCIO, MEIO E FIM)

**Para** que um computador **repita** uma **tarefa** sem **travar** o sistema, devemos respeitar **três etapas lógicas** indispensáveis:

- **Fim** (O Passo de Atualização): A **alteração** do **estado** para que, **eventualmente**, o sistema consiga **sair da repetição**. (Ex: Adicionar +1 ao mês). Sem essa etapa de **fechamento**, o código ficaria preso no "meio" **eternamente**.

# APRESENTANDO O COMANDO WHILE

Em **Python**, a palavra **reservada** para criar **repetições** é o **while** (que significa "enquanto").

# APRESENTANDO O COMANDO WHILE

```
while <condição> :
    <Bloco de Código>
```

# APRESENTANDO O COMANDO WHILE

O **while** funciona **quase** como um comando **if**, mas com **insistência**:

- Se a **condição** for **Falsa**: Ele **ignora** o **bloco de código**, **exatamente** como o if faria.

# APRESENTANDO O COMANDO WHILE

O **while** funciona **quase** como um comando **if**, mas com **insistência**:

- Se a **condição** for **Verdadeira**:
  - Ele **executa** o bloco de **código**, **mas** em vez de ir para a **próxima linha** do programa, ele **volta ao topo** e **testa** a **condição novamente**.
  - Ele repete essa **verificação indefinidamente** até que a **condição** se torne **Falsa**.

# RESOLVENDO O PROBLEMA DA FÁBRICA NA PRÁTICA

**Traduzindo** a lógica para a linguagem **Python**, utilizamos o comando **while** (que significa "**enquanto**").
Veja como o problema da **fábrica** é resolvido **iterativamente**, sem depender da **fórmula** da **P.A.**:

# RESOLVENDO O PROBLEMA DA FÁBRICA NA PRÁTICA

```
1. # 1. INÍCIO
2. mes_atual = 1
3. producao_total = 0
4. capacidade_mes = 100
```

# RESOLVENDO O PROBLEMA DA FÁBRICA NA PRÁTICA

```
1. # 2. MEIO (Condição e Ação)
2. while mes_atual <= 50:
3.     producao_total += capacidade_mes
4.     capacidade_mes += 20
5.     # A fábrica cresce
```

Dica: Escrever valor += 10 é o mesmo que valor = valor + 10

# RESOLVENDO O PROBLEMA DA FÁBRICA NA PRÁTICA

```
    # 3. FIM (Atualização para sair do laço)
    mes_atual = mes_atual + 1

print("Ao final de 50 meses, a produção total foi de:", producao_total)
```

# EXEMPLOS COM O WHILE

**Validação Básica de Senha**

```
senha = input("Digite a senha: ")

# Fica preso no laço enquanto a senha estiver errada
while senha != "1234":
    print("Senha incorreta!")
    senha = input("Tente novamente: ")

print("Acesso liberado!")
```

# EXEMPLOS COM O WHILE

**Simulações Financeiras:**

1. saldo = 1000.0
2. taxa_juros = 0.008  # 0.8% ao mês
3. meses = 0

# EXEMPLOS COM O WHILE

```
# Repete ENQUANTO o saldo não atingir o objetivo
while saldo < 1500.0:
    saldo = saldo + (saldo * taxa_juros)
    meses = meses + 1

print(f"Objetivo alcançado em {meses} meses. Saldo final: R$ {saldo:.2f}")
```

# DÚVIDAS???